

FaceMaskDetection Code WorkFlow

train_mask_detector.py

1. Collecting images for training
2. For each image :
3. Getting the **folder** names(Label- Mask/Without Mask) of the images out from image_paths(which is the output)
4. Converting input images into array
5. Array is then preprocessed using preprocess_input method from mobilenet_v2 (pre-trained model).The pre-trained model's preprocess method is used to avoid issues.
6. Preprocessed Input is converted into array of datatype float32
7. Converting output labels into array
8. Converting text labels ("mask" and "Without mask") into one-hot encoded vectors as neural networks cannot understand text
9. train set and test set are separated
10. Using ImageDataGenerator(DataAugmentation), multiplying the datasets to get more data to train for efficient learning
11. mobilenet_v2 (pre-trained model) is called for transfer learning by changing out input shape to (224,224,3) which is pre-trained model's input shape
12. Output of pre-trained model is used as the input to the actual layers.Here only weights from pre-trained model are used, which has only the features extraction by removing background.
13. AveragePooling of (7,7) - Average of 7*7 matrix sliding window

14. Flattened layer is used to flatten the data to neural network
15. Dense layer(Fully connected layer), Dropout and output layers are used
16. Transfer learning is happening- `model = Model(inputs=baseModel.input, outputs=headModel)`
 `Model()` is a class from the Keras Functional API used to define an entire neural network container by simply pointing to its starting point (input) and ending point (output).
 `inputs=baseModel.input`- It tells Keras that when a new image is fed into the final network, it must enter through the very first layer of the pre-trained base model.
 `outputs=headModel`-It tells Keras that the data must flow completely through the base model, pass straight into your custom head layers, and exit out of the final layer of your head model (where one-hot encoded predictions come out).
17. the base model layers are freezed(`layer.trainable = False`) to stop retraining the layers which are trained already in pre-trained model
18. model is getting compiled with optimizer , accuracy and loss
19. Model is getting trained using fit method with BatchSize, epochs, learningrate
20. Model evaluation by predict method
21. Using `np.argmax`, the predicted values are found
22. classification report is generated
23. Best Model is saved at the end

detect_mask_video.py

1.creating a function **`detect_and_predict_mask`**(frame, faceNet, maskNet):

where frame - frames of video, faceNet - pre-trained model to identify and crop only the face from the whole image , maskNet- our own model

2.(h, w) = frame.shape[:2]

This line extracts the height (h) and width (w) of the incoming video frame. these dimensions are used to calculate the bounding boxes when drawing rectangles around faces.

3.A raw video frame is resized, altered in color, and scaled. The function cv2.dnn.blobFromImage handles all of this preprocessing at once, outputting a highly structured 4D matrix called a “blob”

4.faceNet.setInput(blob)

This sets the input data for the neural network.

5.detections = faceNet.forward()

the forward propagation algorithm is triggered to make predictions

6.for i in range(0, detections.shape[2]):

This loop inspects fixed number of potential faces one by one.

 confidence = detections[0, 0, i, 2]

 This extracts the AI's confidence score for the current object index i.

 A score of 0.92 means the AI is 92% sure it found a face.

7.if confidence > args["confidence"]:

This ignores anything below your minimum threshold (which is set to 0.5 in argparse settings). If the AI is only 20% sure, it skips it.

8.box = detections[0, 0, i, 3:7] * np.array([w, h, w, h])

The face detector outputs coordinates as normalized percentages between 0.0 and 1.0 (e.g., 0.5 means the exact middle of the screen). To draw a box or crop the image, actual pixel positions are needed.

Multiplying those percentages by the frame's original width (w) and height (h) converts them back into real pixel numbers.

9.(startX, startY, endX, endY) = box.astype("int")

Converts those floating-point pixel positions into sharp integers so the code can use them to crop the image.

10.(startX, startY) = (max(0, startX), max(0, startY)) and min(w - 1, endY)
This is a safety check called clipping. If a face is partially cut off at the edge of the webcam frame, the coordinates might accidentally calculate to negative numbers (like -5) or numbers larger than the screen width. This forces the coordinates to stay safely inside the image boundaries, preventing your program from crashing.

11.make a predictions using maskNet.predict() if at least one face was detected

12.return locations and their predictions to the calling function

13.Initializing script's Command Line Interface (CLI). Building a parser, registers three specific configurable flags (--face, --model, and --confidence), and converts the inputs into a clean Python dictionary.

14.loading pre-trained face detector model from disk using two files "deploy.prototxt" and "res10_300x300_ssd_iter_140000.caffemodel"] using the function cv2.dnn.readNet which initializes the OpenCV Deep Learning engine

15.Loading maskNet model from disk

16.Initialising the video stream, wake-up signal to the webcam (device 0) is sent. And adding sleep for 2 seconds ensures that, by the time the code asks for the first frame, the webcam is fully awake, calibrated, and feeding valid image data.

17.While loop over the frames from the video stream starts

- grab the frame from the threaded video stream and resize it to have a maximum width of 400 pixels using imutils.resize(frame, width=400)

- Call function **detect_and_predict_mask(frame, faceNet, maskNet)**
- Start the for loop over the detected face locations and their corresponding locations
- Unpack box(location) variable to get startX, startY, endX, endY values
- Get values for mask and withoutMask from pred
- determine the class label(Mask/WihtoutMask) and the color used to draw the bounding box and text
- Based on the label, display bounding box rectangle on the output frame
- Break the loop if the `q` key was pressed

